

Guia passo a passo – Pipeline de Machine Learning com Penguins

Este guia explica, **em linguagem simples**, tudo o que o código do projeto Penguins está fazendo, **na ordem correta**, para que você consiga entender o fluxo completo sem depender de copiar código às cegas.

Visão geral do objetivo

O objetivo do projeto é:

1. Usar dados **limpos** de pinguins para **treinar um modelo de classificação**
2. Avaliar se o modelo é confiável
3. Usar o modelo final para **prever o sexo** de pinguins onde essa informação está faltando
4. Reconstruir um **dataset completo**, sem valores ausentes

Em termos de Machine Learning:

- **Problema:** classificação binária (female / male)
 - **Tipo:** supervised learning
-

Etapa 1 – Importações

Nesta parte, você importa todas as bibliotecas necessárias:

- `pandas`, `numpy`: manipulação de dados
- `sklearn.preprocessing`: escaladores e codificadores
- `sklearn.model_selection`: divisão treino/teste e validação cruzada

- `sklearn.pipeline`: encadear passos (scaler + modelo)
- `sklearn.metrics`: métricas de avaliação

👉 Nada é treinado aqui. Você só está preparando ferramentas.

Etapa 2 – Carregar o dataset limpo

Python ▾

```
1 penguins_clean = pd.read_csv('./data/penguins_clean.csv')  
2
```

Esse arquivo contém:

- Apenas linhas **sem valores ausentes**
- Apenas observações onde o sexo é conhecido

👉 **Regra importante:** modelos supervisionados só podem ser treinados com dados onde o alvo (`sex`) é conhecido.

Etapa 3 – Codificação de variáveis categóricas

Por que isso é necessário?

Modelos de ML não entendem texto, apenas números.

O que o código faz

- `species` (categórica) → `species_enc`
- `sex` (alvo) → `sex_enc`

Plain Text ▾

```
1 female → 0
2 male   → 1
3
```

👉 `LabelEncoder` cria essa conversão.

⚠️ Importante:

- Codificar o **alvo** com `LabelEncoder` é correto
- Para aprendizado, usar `LabelEncoder` em `species` é aceitável neste contexto didático

Etapa 4 – Separar features (X) e target (y)

Python ▾

```
1 X = df[feature_cols]
2 y = df['sex_enc']
3
```

O que são features?

Variáveis usadas para prever o sexo:

- medidas do corpo
- espécie (codificada)

O que é o target?

- `sex_enc` → o que queremos prever

👉 Esta separação é **fundamental** em ML.

Etapa 5 – Train / Test Split

Python ▾

```
1 X_train, X_test, y_train, y_test = train_test_split(...)
2
```

O que isso faz

- Divide os dados em dois conjuntos:
 - **Treino (80%)** → ensinar o modelo
 - **Teste (20%)** → avaliar se ele generaliza
- `stratify=y` garante que a proporção male/female seja mantida

👉 Isso cria um **teste honesto**, nunca visto pelo modelo.

Etapa 6 – Construção do Pipeline

O pipeline define uma **linha de montagem**:

1. Escalonar os dados (`StandardScaler`)
2. Treinar um modelo (`RandomForestClassifier`)

Plain Text ▾

```
1 X → scaler → modelo → previsão
2
```

Por que usar pipeline?

- Evita vazamento de dados
- Garante que treino e teste passam pelos mesmos passos
- Facilita reaproveitamento

👉 Tudo fica organizado em um único objeto.

Etapa 7 – Treinamento do modelo

Python ▾

```
1 pipeline.fit(X_train, y_train)
2
```

O que acontece aqui:

- O scaler é ajustado **somente no treino**
- O Random Forest aprende padrões entre medidas e sexo

👉 A partir daqui, o modelo está treinado.

Etapa 8 – Avaliação no conjunto de teste

Previsão

Python ▾

```
1 y_pred = pipeline.predict(X_test)
2
```

Métricas

- `classification_report`
 - precision
 - recall

- f1-score
- `confusion_matrix`
 - mostra erros e acertos por classe

👉 Esta etapa responde: “posso confiar nesse modelo?”

Etapa 9 – Validação cruzada (cross-validation)

Python ▾

```
1 cross_val_score(...)  
2
```

O que isso faz

- Divide o dataset limpo em 5 partes
- Treina 5 vezes
- Cada parte vira teste uma vez

Resultado:

- média da acurácia
- desvio padrão

👉 Isso mede **estabilidade**, não só desempenho.

Etapa 10 – Importância das variáveis

Python ▾

```
1 pipeline.named_steps['clf'].feature_importances_  
2
```

Como o modelo é Random Forest, ele consegue dizer:

- quais variáveis mais ajudam a decidir o sexo

O gráfico mostra:

- maior barra = maior influência

👉 Isso ajuda a **interpretar** o modelo.

Etapa 11 – Prever sexo dos pinguins desconhecidos

Aqui entra o **uso prático** do modelo.

O que acontece

1. Carrega `penguins_unknown_sex.csv`
2. Codifica `species` com o **mesmo encoder**
3. Usa o pipeline treinado para:
 - prever o sexo
 - calcular a confiança

👉 O modelo agora está sendo usado para **inferência**.

Etapa 12 – Reconstruir o dataset completo

Python ▾

```
1 penguins_full = pd.concat(...)  
2
```

- Junta:

- dados limpos
- 9 linhas agora preenchidas

Verificações finais:

- número de linhas
- contagem de valores nulos

👉 Resultado: dataset final **sem NaNs**.

Etapa 13 – Salvamento

Python ▾

```
1 penguins_full.to_csv('penguins_full.csv')  
2
```

O projeto termina com:

- modelo treinado
 - previsões feitas
 - dataset completo salvo
-

Resumo em uma frase

Você treinou um modelo supervisionado com dados limpos, validou seu desempenho, usou esse modelo para prever valores ausentes e reconstruiu um dataset completo de forma correta e reproduzível.

Se quiser, posso:

- simplificar isso ainda mais
- transformar em texto de relatório
- explicar cada etapa **como se fosse uma prova oral**
- ou ligar cada parte aos conceitos teóricos da disciplina